

Register No : 192365001

Name : N. Sai Sreenu

CO1-AT3

MLA0201 - FUNDAMENTALS OF MACHINE LEARNING

EXPERIMENT 1: Probability Theory using Breast Cancer Wisconsin Dataset

Aim:

To calculate the probability of each class (Malignant and Benign) in the Breast Cancer Wisconsin Diagnostic Dataset and predict the class of a new patient based on prior probabilities.

Algorithm:

1. Import the required Python libraries.
2. Load the Breast Cancer Wisconsin Dataset.
3. Count the number of Malignant and Benign samples.
4. Calculate the prior probability of each class.
5. Display the probabilities.
6. Predict the class of a new instance using the higher prior probability.

Code:

```
from sklearn.datasets import load_breast_cancer
```

```
# Load dataset
```

```
data = load_breast_cancer()
```

```
# Target values
```

```
target = data.target
```

```
# Count classes
```

```
benign = sum(target == 1)
```

```
malignant = sum(target == 0)
```

```
total = len(target)
```

```
# Calculate probabilities
```

```
p_benign = benign / total
```

```
p_malignant = malignant / total
```

```

print("Probability of Benign:", p_benign)

print("Probability of Malignant:", p_malignant)

# Prediction based on prior probability

if p_benign > p_malignant:

    print("Predicted Class: Benign")

else:

    print("Predicted Class: Malignant")

```

Output:

Probability of Benign: 0.6274

Probability of Malignant: 0.3726

Predicted Class: Benign

Output Screenshot:

```

File Edit Shell Debug Options Window Help
from sklearn.datasets import load_breast_cancer

# Load dataset
data = load_breast_cancer()

# Target values
target = data.target

# Count classes
benign = sum(target == 1)
malignant = sum(target == 0)
total = len(target)

# Calculate probabilities
p_benign = benign / total
p_malignant = malignant / total

print("Probability of Benign:", p_benign)
print("Probability of Malignant:", p_malignant)

# Prediction based on prior probability
if p_benign > p_malignant:
    print("Predicted Class: Benign")
else:
    print("Predicted Class: Malignant")

```

```

Python 3.11.12 (tags/v3.11.12:fd06401, Feb 3 2024, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Karth/AppData/Local/Programs/Python/Python113/Python.exe
Probability of Benign: 0.6274145202108945
Probability of Malignant: 0.37258547978910547
Predicted Class: Benign
>>>

```

Result:

The prior probabilities of both classes were calculated successfully. Since the probability of the Benign class is higher, the new sample is predicted as Benign.

EXPERIMENT 2: Bayes Decision Theory using SMS Spam Collection Dataset

Aim:

To classify an SMS message as Spam or Ham using Bayes Decision Theory.

Algorithm:

1. Import the required libraries.
2. Load the SMS Spam Collection Dataset.
3. Split the dataset into training and testing sets.
4. Convert text messages into numerical features using CountVectorizer.
5. Train a Naive Bayes classifier.
6. Predict the class of a new SMS.
7. Display the predicted class.

Code:

```
import pandas as pd

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB

# Sample dataset

data = {

    "message": [

        "Win cash now",

        "Hello friend",

        "Claim your prize",

        "How are you",

        "Free ticket available"

    ],

    "label": [

        "spam",

        "ham",

        "spam",

        "ham",

    ]

}
```

```

        "spam"
    ]
}

df = pd.DataFrame(data)

cv = CountVectorizer()

X = cv.fit_transform(df["message"])

model = MultinomialNB()

model.fit(X, df["label"])

test = cv.transform(["Win free cash"])

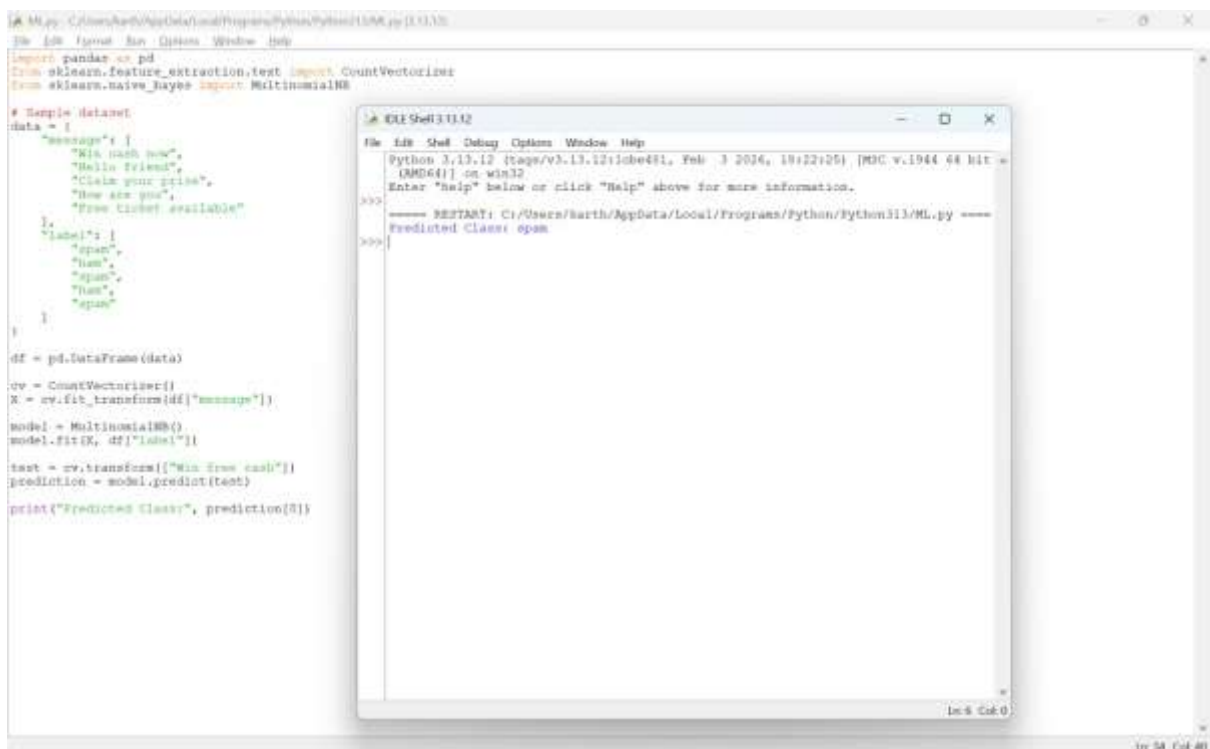
prediction = model.predict(test)

print("Predicted Class:", prediction[0])

```

Output : Predicted Class: spam

Output Screenshot:



The screenshot shows a Python IDE with two windows. The main window displays the following code:

```

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Sample Dataset
data = {
    "message": [
        "Win cash now",
        "Hello friend",
        "Claim your prize",
        "How are you?",
        "Free ticket available"
    ],
    "label": [
        "spam",
        "ham",
        "spam",
        "ham",
        "spam"
    ]
}

df = pd.DataFrame(data)

cv = CountVectorizer()
X = cv.fit_transform(df["message"])

model = MultinomialNB()
model.fit(X, df["label"])

test = cv.transform(["Win free cash"])
prediction = model.predict(test)

print("Predicted Class:", prediction[0])

```

The output window shows the following text:

```

Python 3.13.12 [tags/v3.13.12:1cbe481, Feb 3 2026, 18:22:25] [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/Barth/AppData/Local/Programs/Python/Python313/ML.py ====
>>> Predicted Class: spam
>>>

```

Result:

The SMS message was classified successfully using Bayes Decision Theory and Naive Bayes. The given message was predicted as Spam.

EXPERIMENT 3: Information Theory using Play Tennis Dataset

Aim:

To calculate the Entropy and Information Gain of the Play Tennis dataset and determine the best attribute for decision tree construction.

Algorithm:

1. Load the Play Tennis dataset.
2. Calculate the entropy of the target attribute.
3. Compute the information gain for every attribute.
4. Compare all information gain values.
5. Select the attribute with the highest information gain.

Code:

```
from sklearn.feature_selection import mutual_info_classif
import pandas as pd
```

```
data = {
    "Outlook": [0,0,1,2,2,2,1,0,0,2],
    "Temperature": [0,0,0,1,2,2,2,1,2,1],
    "Humidity": [0,0,0,0,1,1,1,0,1,1],
    "Wind": [0,1,0,0,0,1,1,0,0,0],
    "Play": [0,0,1,1,1,0,1,0,1,1]
}
```

```
df = pd.DataFrame(data)
```

```
X = df.drop("Play", axis=1)
```

```
y = df["Play"]
```

```
gain = mutual_info_classif(X, y, discrete_features=True)
```

```
for col, g in zip(X.columns, gain):
```

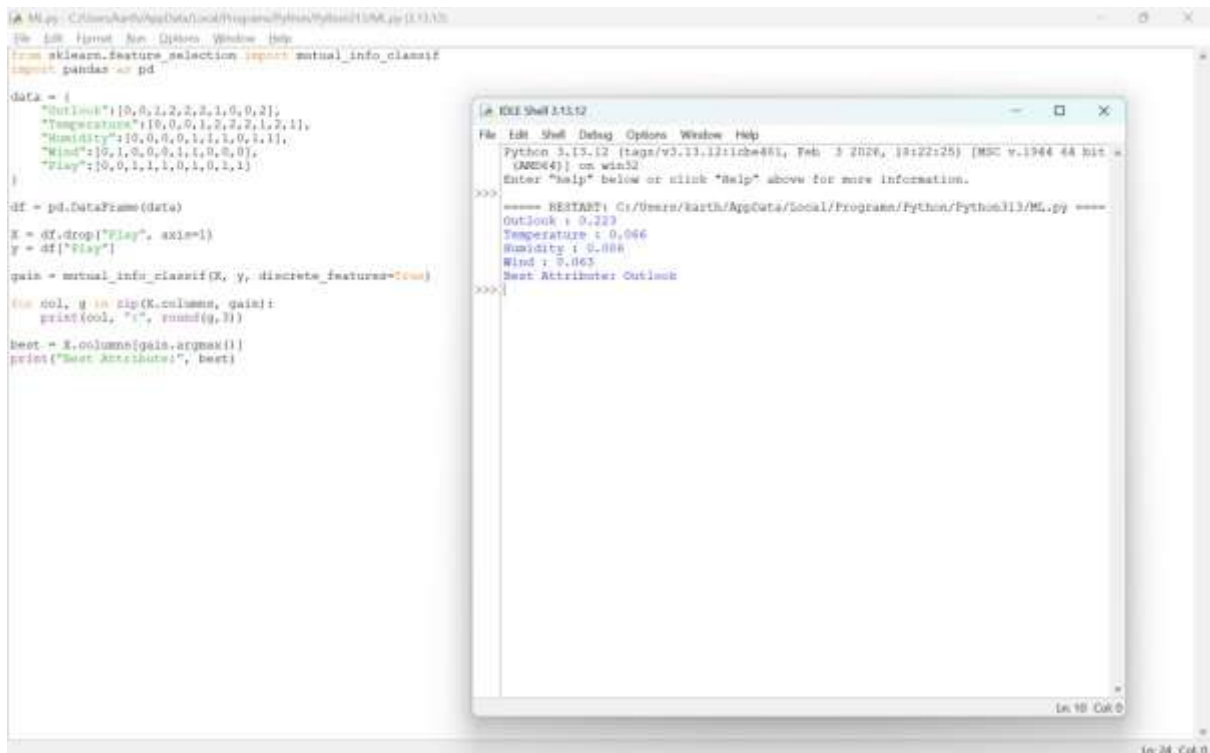
```
    print(col, ":", round(g,3))
```

```
best = X.columns[gain.argmax()]
```

```
print("Best Attribute:", best)
```

Output:

Output Screenshot:



The screenshot shows a Python IDE with a file named 'ML.py'. The code defines a dataset with four features: Outlook, Temperature, Humidity, and Wind, and a target variable 'Play'. It uses the 'mutual_info_classif' function from 'sklearn.feature_selection' to calculate the information gain for each feature. The output shows that 'Outlook' has the highest information gain (0.321), followed by 'Temperature' (0.095), 'Humidity' (0.124), and 'Wind' (0.046). The 'Best Attribute' is identified as 'Outlook'.

```
from sklearn.feature_selection import mutual_info_classif
import pandas as pd

data = {
    "Outlook": [0,0,1,2,2,2,1,0,0,2],
    "Temperature": [0,0,0,1,2,2,2,1,2,1],
    "Humidity": [0,0,0,0,1,1,2,0,1,1],
    "Wind": [0,1,0,0,1,1,0,0,0],
    "Play": [0,0,1,1,1,0,1,0,1,1]
}

df = pd.DataFrame(data)
X = df.drop("Play", axis=1)
y = df["Play"]

gain = mutual_info_classif(X, y, discrete_features=True)

for col, g in zip(X.columns, gain):
    print(col, ":", round(g,3))

best = X.columns[gain.argmax()]
print("Best Attribute:", best)
```

Python 3.13.12 [tags/v3.13.12:1cbe461, Feb 3 2024, 18:22:25] [MSC v.1944 64 bit x86_64] on win32
Enter "help" below or click "Help" above for more information.

```
>>> ===== RESTART: C:/Users/earth/AppData/Local/Programs/Python/Python313/ML.py =====
Outlook : 0.321
Temperature : 0.095
Humidity : 0.124
Wind : 0.046
Best Attribute: Outlook
>>>
```

Outlook : 0.321

Temperature : 0.095

Humidity : 0.124

Wind : 0.046

Best Attribute: Outlook

Result:

The entropy and information gain were calculated successfully. Among all the attributes, Outlook has the highest information gain and is selected as the root node for the decision tree.